

Programació iterativa metòdica

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Politècnica Superior d'Enginyeria
de Vilanova i la Geltrú

Índex

- 1 Programes iteratius
- 2 Correctesa de programes
 - Correctesa de programes iteratius
- 3 Justificació de la correctesa de programes iteratius
 - Exemples de justificació de la correctesa
- 4 Exercici

Índex

- 1 Programes iteratius
- 2 Correctesa de programes
- 3 Justificació de la correctesa de programes iteratius
- 4 Exercici

Programes iteratius

- Esquema bàsic d'un programa iteratiu:

```

/* Pre */
inicialitzacions
while (B) {
    S
}
...
/* Post */

```

Programes iteratius

- Esquema bàsic d'un programa iteratiu:

```

/* Pre */
inicialitzacions
while (B) {
    S
}
...
/* Post */

```

- **inicialitzacions:** Conjunt d'instruccions que donen valors inicials a les variables. Podria ser $\emptyset$.

Programes iteratius

- Esquema bàsic d'un programa iteratiu:

```

/* Pre */
inicialitzacions
while (B) {
    S
}
...
/* Post */

```

- **inicialitzacions**: Conjunt d'instruccions que donen valors inicials a les variables. Podria ser $\emptyset$.
- **B**: Condició d'entrada de la iteració o bucle.

Programes iteratius

- Esquema bàsic d'un programa iteratiu:

```

/* Pre */
inicialitzacions
while (B) {
    S
}
...
/* Post */

```

- **inicialitzacions**: Conjunt d'instruccions que donen valors inicials a les variables. Podria ser $\emptyset$.
- **B**: Condició d'entrada de la iteració o bucle.
- **S**: Conjunt d'instruccions que s'executen a cada iteració.

Índex

- 1 Programes iteratius
- 2 Correctesa de programes
 - Correctesa de programes iteratius
- 3 Justificació de la correctesa de programes iteratius
- 4 Exercici

Correctesa de programes

- Què és un programa correcte?

Correctesa de programes

- Què és un programa correcte?
- Com es demostra que un programa és correcte?

Correctesa de programes

- Què és un programa correcte?
- Com es demostra que un programa és correcte?
 - Validació dinàmica (jocs de proves).

Correctesa de programes

- Què és un programa correcte?
- Com es demostra que un programa és correcte?
 - Validació dinàmica (jocs de proves).
 - Validació estàtica (verificació).

Correctesa de programes

- Què és un programa correcte?
- Com es demostra que un programa és correcte?
 - Validació dinàmica (jocs de proves).
 - Validació estàtica (verificació).
- En aquest tema ens centrarem en la correctesa de programes iteratius.

Índex

1 Programes iteratius

2 Correctesa de programes

- Correctesa de programes iteratius

3 Justificació de la correctesa de programes iteratius

- Exemples de justificació de la correctesa

4 Exercici

Principi d'inducció: definició (recordatori)

- Per demostrar que un programa és correcte amb un nombre finit de proves cal un mecanisme d'inducció.

Principi d'inducció: definició (recordatori)

- Per demostrar que un programa és correcte amb un nombre finit de proves cal un mecanisme d'inducció.
- El Principi d'Inducció (matemàtiques) serveix per demostrar propietats que valen per tot nombre natural.

Principi d'inducció: definició (recordatori)

- Per demostrar que un programa és correcte amb un nombre finit de proves cal un mecanisme d'inducció.
- El Principi d'Inducció (matemàtiques) serveix per demostrar propietats que valen per tot nombre natural.

Principi d'inducció (en els naturals)

P és una propietat que es pot satisfer o no per cada nombre natural.

- 1 Si el número 0 compleix la propietat P , i
- 2 quan un nombre natural n compleix la propietat P , aleshores també la compleix el nombre següent $n + 1$

aleshores tots els nombres naturals compleixen la propietat P .

Principi d'inducció i programes iteratius

- Fem servir el principi d'inducció per demostrar que es compleix una propietat P (en els naturals).

Principi d'inducció i programes iteratius

- Fem servir el principi d'inducció per demostrar que es compleix una propietat P (en els naturals).
- Què és P en un programa iteratiu?

Principi d'inducció i programes iteratius

- Fem servir el principi d'inducció per demostrar que es compleix una propietat P (en els naturals).
- Què és P en un programa iteratiu? La seva correctesa.

Principi d'inducció i programes iteratius

- Fem servir el principi d'inducció per demostrar que es compleix una propietat P (en els naturals).
- Què és P en un programa iteratiu? La seva correctesa.
- Com apliquem el principi d'inducció per demostrar que un programa iteratiu és correcte?

Principi d'inducció i programes iteratius

- Fem servir el principi d'inducció per demostrar que es compleix una propietat P (en els naturals).
- Què és P en un programa iteratiu? La seva correctesa.
- Com apliquem el principi d'inducció per demostrar que un programa iteratiu és correcte?
 - ❶ Demostrar que si a l'inici de l'execució les variables satisfan la precondició, en acabar l'execució satisfan la postcondició.

Principi d'inducció i programes iteratius

- Fem servir el principi d'inducció per demostrar que es compleix una propietat P (en els naturals).
- Què és P en un programa iteratiu? La seva correctesa.
- Com apliquem el principi d'inducció per demostrar que un programa iteratiu és correcte?
 - ❶ Demostrar que si a l'inici de l'execució les variables satisfan la precondició, en acabar l'execució satisfan la postcondició.
 - ❷ Demostrar que l'execució acabarà (no estem en un bucle infinit).

Demostrar la correctesa dels programes iteratius

Pas 1 Trobar l'**invariant**: propietat que es manté a cada iteració.

Demostrar la correctesa dels programes iteratius

Pas 1 Trobar l'**invariant**: propietat que es manté a cada iteració.

Pas 2 Provar que l'invariant trobat és invariant.

Demostrar la correctesa dels programes iteratius

Pas 1 Trobar l'**invariant**: propietat que es manté a cada iteració.

Pas 2 Provar que l'invariant trobat és invariant.

Pas 2.1 Ha de ser cert abans de començar a iterar.

Demostrar la correctesa dels programes iteratius

Pas 1 Trobar l'**invariant**: propietat que es manté a cada iteració.

Pas 2 Provar que l'invariant trobat és invariant.

Pas 2.1 Ha de ser cert abans de començar a iterar.

Pas 2.2 Si s'entra al bucle (B és certa), després d'executar les seves instruccions una vegada (S), l'invariant segueix sent cert.

Demostrar la correctesa dels programes iteratius

Pas 1 Trobar l'**invariant**: propietat que es manté a cada iteració.

Pas 2 Provar que l'invariant trobat és invariant.

Pas 2.1 Ha de ser cert abans de començar a iterar.

Pas 2.2 Si s'entra al bucle (B és certa), després d'executar les seves instruccions una vegada (S), l'invariant segueix sent cert.

Pas 2.3 Quan se surt del bucle (B és falsa), sabent que l'invariant segueix sent cert, ja s'ha de complir la $Post$.

Demostrar la correctesa dels programes iteratius

Pas 1 Trobar l'**invariant**: propietat que es manté a cada iteració.

Pas 2 Provar que l'invariant trobat és invariant.

Pas 2.1 Ha de ser cert abans de començar a iterar.

Pas 2.2 Si s'entra al bucle (B és certa), després d'executar les seves instruccions una vegada (S), l'invariant segueix sent cert.

Pas 2.3 Quan se surt del bucle (B és falsa), sabent que l'invariant segueix sent cert, ja s'ha de complir la $Post$.

Pas 3 Trobar una **funció de fita**: funció sobre una o més variables del bucle que tingui un valor **natural** i que decreixi a cada iteració.

Passos

```

/* Pre */
Inicialitzacions
while (B) {
    S
}
/* Post */
    
```

Passos

```

/* Pre */
Inicialitzacions
while (B) {
    S
}
/* Post */
    
```

1 $\{Pre\}$ Inicialitzacions $\{Invariant\}$

Passos

```

/* Pre */
Inicialitzacions
while (B) {
    S
}
/* Post */

```

- 1 $\{Pre\}$ Inicialitzacions $\{Invariant\}$
- 2 $\{Invariant \text{ and not } B\} \implies \{Post\}$

Passos

```

/* Pre */
Inicialitzacions
while (B) {
    S
}
/* Post */

```

- 1 $\{Pre\}$ Inicialitzacions $\{Invariant\}$
- 2 $\{Invariant \text{ and not } B\} \implies \{Post\}$
- 3 $\{Invariant \text{ and } B\} S \{Invariant\}$

Passos

```

/* Pre */
Inicialitzacions
while (B) {
    S
}
/* Post */

```

- 1 $\{Pre\}$ Inicialitzacions $\{Invariant\}$
- 2 $\{Invariant \text{ and not } B\} \implies \{Post\}$
- 3 $\{Invariant \text{ and } B\} S \{Invariant\}$
- 4 $Invariant \implies t \in \mathbb{N}$

Passos

```

/* Pre */
Inicialitzacions
while (B) {
    S
}
/* Post */

```

- 1 $\{Pre\}$ Inicialitzacions $\{Invariant\}$
- 2 $\{Invariant \text{ and not } B\} \implies \{Post\}$
- 3 $\{Invariant \text{ and } B\} S \{Invariant\}$
- 4 $Invariant \implies t \in \mathbb{N}$
- 5 $\{Invariant \text{ and } B \text{ and } t = T\} S \{t < T\}$

Índex

- 1 Programes iteratius
- 2 Correctesa de programes
- 3 Justificació de la correctesa de programes iteratius
 - Exemples de justificació de la correctesa
- 4 Exercici

Restriccions del codi en C++

Per poder fer les justificacions de correctesa, als nostres codis:

- 1 Només fem iteracions amb la instrucció `while` (no `for`).

Restriccions del codi en C++

Per poder fer les justificacions de correctesa, als nostres codis:

- ❶ Només fem iteracions amb la instrucció `while` (no `for`).
- ❷ Les expressions booleanes no s'avaluen amb prioritat.

Restriccions del codi en C++

Per poder fer les justificacions de correctesa, als nostres codis:

- ❶ Només fem iteracions amb la instrucció `while` (no `for`).
- ❷ Les expressions booleanes no s'avaluen amb prioritat.
- ❸ Per les funcions, usem un únic `return` que serà la darrera instrucció de la funció.

Restriccions del codi en C++

Per poder fer les justificacions de correctesa, als nostres codis:

- 1 Només fem iteracions amb la instrucció `while` (no `for`).
- 2 Les expressions booleanes no s'avaluen amb prioritat.
- 3 Per les funcions, usem un únic `return` que serà la darrera instrucció de la funció.

Si el codi que tenim no compleix aquestes restriccions podem transformar-lo fàcilment en un d'equivalent que sí que les compleixi.

Passos d'una justificació

- 1 *Inicialitzacions*: Demostrar que les inicialitzacions fan cert l'invariant abans d'entrar al bucle

$\{Pre\}$ Inicialitzacions $\{Invariant\}$

Passos d'una justificació

- 1 *Inicialitzacions*: Demostrar que les inicialitzacions fan cert l'invariant abans d'entrar al bucle

$$\{Pre\} \text{ Inicialitzacions } \{Invariant\}$$

- 2 *Condició de sortida*: Demostrar que a partir de l'invariant i la condició de sortida, se satisfà la $Post$

$$\{Invariant \text{ and not } B\} \implies \{Post\}$$

Si hi ha instruccions a la sortida del bucle, demostrar

$$\{Invariant \text{ and not } B\} \text{ Instruccions finals } \{Post\}$$

Passos d'una justificació

- 1 **Inicialitzacions:** Demostrar que les inicialitzacions fan cert l'invariant abans d'entrar al bucle

$$\{Pre\} \text{ Inicialitzacions } \{Invariant\}$$

- 2 **Condició de sortida:** Demostrar que a partir de l'invariant i la condició de sortida, se satisfà la $Post$

$$\{Invariant \text{ and not } B\} \implies \{Post\}$$

Si hi ha instruccions a la sortida del bucle, demostrar

$$\{Invariant \text{ and not } B\} \text{ Instruccions finals } \{Post\}$$

- 3 **Cos del bucle:** Demostrar que l'execució de les instruccions mantenen cert l'invariant

$$\{Invariant \text{ and } B\} S \{Invariant\}$$

Passos d'una justificació

- 1 **Inicialitzacions:** Demostrar que les inicialitzacions fan cert l'invariant abans d'entrar al bucle

$$\{Pre\} \text{ Inicialitzacions } \{Invariant\}$$

- 2 **Condició de sortida:** Demostrar que a partir de l'invariant i la condició de sortida, se satisfà la $Post$

$$\{Invariant \text{ and not } B\} \implies \{Post\}$$

Si hi ha instruccions a la sortida del bucle, demostrar

$$\{Invariant \text{ and not } B\} \text{ Instruccions finals } \{Post\}$$

- 3 **Cos del bucle:** Demostrar que l'execució de les instruccions mantenen cert l'invariant

$$\{Invariant \text{ and } B\} S \{Invariant\}$$

- 4 **Acabament:** Demostrar que la funció fita $\in \mathbb{N}$ i que decreix a cada iteració

$$Invariant \implies t \in \mathbb{N}$$

$$\{Invariant \text{ and } B \text{ and } t = T\} S \{t < T\}$$

Índex

- 1 Programes iteratius
- 2 Correctesa de programes
 - Correctesa de programes iteratius
- 3 Justificació de la correctesa de programes iteratius
 - Exemples de justificació de la correctesa
- 4 Exercici

Exemple 1: calcular l'altura d'una pila

- Donada una pila d'enters, calcular la seva altura o nombre d'elements.

Exemple 1: calcular l'altura d'una pila

- Donada una pila d'enters, calcular la seva altura o nombre d'elements.
- Especificació:

```
int altura_pila_int(stack<int> p);
/* Pre: p = P */
/* Post: El resultat és el nombre d'elements de P */
```

Exemple 1: calcular l'altura d'una pila

- Donada una pila d'enters, calcular la seva altura o nombre d'elements.
- Especificació:

```
int altura_pila_int(stack<int> p);
/* Pre: p = P */
/* Post: El resultat és el nombre d'elements de P */
```

Noteu que usem P per referir-nos al valor inicial de p , atès que aquesta es modifica durant l'execució de la funció.

Solució: calcular l'altura d'una pila

```
int altura_pila_int(stack<int> p)
/* Pre: p = P */
/* Post: El resultat és el nombre d'elements de P */
{
    int n = 0;

    /* Inv: ? */
    while (not p.empty()) {
        ++n;
        p.pop();
    }
    return n;
}
```

Solució: calcular l'altura d'una pila

```

int altura_pila_int(stack<int> p)
/* Pre: p = P */
/* Post: El resultat és el nombre d'elements de P */
{
    int n = 0;

    /* Inv: n="nombre d'elems. de la part tractada de P"
    i p="elements de P que queden per tractar" */
    while (not p.empty()) {
        ++n;
        p.pop();
    }
    return n;
}

```

Justificació informal: calcular l'altura d'una pila

❶ *Inicialitzacions:*

Justificació informal: calcular l'altura d'una pila

- 1 *Inicialitzacions*: Inicialment considerem que no hi ha cap element de P tractat (p conté tots els elements). Amb $n = 0$ satisfem l'invariant.

Justificació informal: calcular l'altura d'una pila

- 1 *Inicialitzacions*: Inicialment considerem que no hi ha cap element de P tractat (p conté tots els elements). Amb $n = 0$ satisfem l'invariant.
- 2 *Condició de sortida*:

Justificació informal: calcular l'altura d'una pila

- 1 *Inicialitzacions:* Inicialment considerem que no hi ha cap element de P tractat (p conté tots els elements). Amb $n = 0$ satisfem l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tota la pila P hagi estat tractada, o sigui, que p no tingui cap element. Sortim quan p sigui buida $\Rightarrow$ ens quedem mentre `not p.empty()`.

Justificació informal: calcular l'altura d'una pila

- 1 *Inicialitzacions:* Inicialment considerem que no hi ha cap element de P tractat (p conté tots els elements). Amb $n = 0$ satisfem l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tota la pila P hagi estat tractada, o sigui, que p no tingui cap element. Sortim quan p sigui buida $\Rightarrow$ ens quedem mentre `not p.empty()`.
- 3 *Cos del bucle:*

Justificació informal: calcular l'altura d'una pila

- 1 *Inicialitzacions:* Inicialment considerem que no hi ha cap element de P tractat (p conté tots els elements). Amb $n = 0$ satisfem l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tota la pila P hagi estat tractada, o sigui, que p no tingui cap element. Sortim quan p sigui buida $\Rightarrow$ ens quedem mentre `not p.empty()`.
- 3 *Cos del bucle:* Amb `++n` i `p.pop()` mantenim l'invariant després d'una iteració del bucle. Podem fer `p.pop()` perquè la condició del bucle ho permet.

Justificació informal: calcular l'altura d'una pila

- 1 *Inicialitzacions:* Inicialment considerem que no hi ha cap element de P tractat (p conté tots els elements). Amb $n = 0$ satisfem l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tota la pila P hagi estat tractada, o sigui, que p no tingui cap element. Sortim quan p sigui buida $\Rightarrow$ ens quedem mentre `not p.empty()`.
- 3 *Cos del bucle:* Amb `++n` i `p.pop()` mantenim l'invariant després d'una iteració del bucle. Podem fer `p.pop()` perquè la condició del bucle ho permet.
- 4 *Acabament:*

Justificació informal: calcular l'altura d'una pila

- 1 *Inicialitzacions:* Inicialment considerem que no hi ha cap element de P tractat (p conté tots els elements). Amb $n = 0$ satisfem l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tota la pila P hagi estat tractada, o sigui, que p no tingui cap element. Sortim quan p sigui buida $\Rightarrow$ ens quedem mentre `not p.empty()`.
- 3 *Cos del bucle:* Amb `++n` i `p.pop()` mantenim l'invariant després d'una iteració del bucle. Podem fer `p.pop()` perquè la condició del bucle ho permet.
- 4 *Acabament:* Agaferem com a funció fita **la mida de p** perquè $\in \mathbb{N}$ i decreix a cada iteració.

Exemple 2: cercar en una cua

- Donada una cua d'enters i un enter, comprovar si l'enter apareix a la cua.

Exemple 2: cercar en una cua

- Donada una cua d'enters i un enter, comprovar si l'enter apareix a la cua.
- Especificació:

```

bool cerca_cua_int(queue<int> c, int x);
/* Pre: c = C */
/* Post: El resultat ens diu si x és un element de C
        o no */
    
```

Exemple 2: cercar en una cua

- Donada una cua d'enters i un enter, comprovar si l'enter apareix a la cua.
- Especificació:

```
bool cerca_cua_int(queue<int> c, int x);
/* Pre: c = C */
/* Post: El resultat ens diu si x és un element de C
        o no */
```

Noteu que usem *C* per referir-nos al valor inicial de *c*, atès que aquesta es modifica durant l'execució de la funció.

Solució parcial: cercar en una cua

```

bool cerca_cua_int(queue<int> c, int x)
/* Pre:  c = C */
/* Post: El resultat ens diu si x és un element de C o no */
{
    bool trobat = false;
    /* Inv: ? */
    while (not c.empty() and not trobat) {
        trobat = (c.front() == x);
        c.pop();
    }
    return trobat;
}
    
```

Solució: cercar en una cua

```

bool cerca_cua_int(queue<int> c, int x)
/* Pre:  c = C */
/* Post: El resultat ens diu si x és un element de C o no */
{
    bool trobat = false;
    /* Inv: trobat="l'element x està a la part tractada de C"
       i c="elements de C que queden per tractar" */
    while (not c.empty() and not trobat) {
        trobat = (c.front() == x);
        c.pop();
    }
    return trobat;
}

```

Justificació informal: cercar en una cua

1 *Inicialitzacions:*

Justificació informal: cercar en una cua

- 1 *Inicialitzacions:* Inicialment no hi ha cap element de C tractat (c conté tots els elements). Amb `trobat = false` fem cert l'invariant.

Justificació informal: cercar en una cua

- 1 *Inicialitzacions:* Inicialment no hi ha cap element de C tractat (c conté tots els elements). Amb `trobat = false` fem cert l'invariant.
- 2 *Condició de sortida:*

Justificació informal: cercar en una cua

- 1 *Inicialitzacions:* Inicialment no hi ha cap element de C tractat (c conté tots els elements). Amb `trobat = false` fem cert l'invariant.
- 2 *Condició de sortida:* Si c és buida o ja hem trobat l'element (*trobat* és cert), aleshores ja tenim la `Post`.

Justificació informal: cercar en una cua

- 1 *Inicialitzacions:* Inicialment no hi ha cap element de C tractat (c conté tots els elements). Amb `trobat = false` fem cert l'invariant.
- 2 *Condició de sortida:* Si c és buida o ja hem trobat l'element (*trobat* és cert), aleshores ja tenim la `Post`.
- 3 *Cos del bucle:*

Justificació informal: cercar en una cua

- 1 *Inicialitzacions:* Inicialment no hi ha cap element de C tractat (c conté tots els elements). Amb `trobat = false` fem cert l'invariant.
- 2 *Condició de sortida:* Si c és buida o ja hem trobat l'element (*trobat* és cert), aleshores ja tenim la `Post`.
- 3 *Cos del bucle:* Actualitzant el valor de *trobat* i avançant en la cua, mantenim l'invariant després d'una iteració del bucle. Podem fer `c.front()` i `c.pop()` perquè la condició del bucle ho permet.

Justificació informal: cercar en una cua

- 1 *Inicialitzacions:* Inicialment no hi ha cap element de C tractat (c conté tots els elements). Amb `trobat = false` fem cert l'invariant.
- 2 *Condició de sortida:* Si c és buida o ja hem trobat l'element (*trobat* és cert), aleshores ja tenim la `Post`.
- 3 *Cos del bucle:* Actualitzant el valor de *trobat* i avançant en la cua, mantenim l'invariant després d'una iteració del bucle. Podem fer `c.front()` i `c.pop()` perquè la condició del bucle ho permet.
- 4 *Acabament:*

Justificació informal: cercar en una cua

- 1 *Inicialitzacions:* Inicialment no hi ha cap element de C tractat (c conté tots els elements). Amb `trobat = false` fem cert l'invariant.
- 2 *Condició de sortida:* Si c és buida o ja hem trobat l'element (*trobat* és cert), aleshores ja tenim la `Post`.
- 3 *Cos del bucle:* Actualitzant el valor de *trobat* i avançant en la cua, mantenim l'invariant després d'una iteració del bucle. Podem fer `c.front()` i `c.pop()` perquè la condició del bucle ho permet.
- 4 *Acabament:* Agafem com a funció fita la mida de c perquè $\in \mathbb{N}$ i decreix a cada iteració.

Exemple 3: sumar k a tots els elements d'una llista

- Donada una llista i un enter k , transformar-la en una altra resultant de sumar k a cada element de la llista original.

Exemple 3: sumar k a tots els elements d'una llista

- Donada una llista i un enter k , transformar-la en una altra resultant de sumar k a cada element de la llista original.
- Especificació:

```

void suma_llista_k(list<int> &l, int k);
/* Pre: l = L */
/* Post: Cada element de l és la suma de k i l'element de L
        a la seva mateixa posició */
    
```

Exemple 3: sumar k a tots els elements d'una llista

- Donada una llista i un enter k , transformar-la en una altra resultant de sumar k a cada element de la llista original.
- Especificació:

```
void suma_llista_k(list<int> &l, int k);
/* Pre: l = L */
/* Post: Cada element de l és la suma de k i l'element de L
a la seva mateixa posició */
```

Noteu que usem L per referir-nos al valor inicial de l , atès que aquesta es modifica durant l'execució de la funció.

Solució: sumar k a tots els elements d'una llista

```
void suma_llista_k(list<int> &l, int k)
/* Pre: l = L */
/* Post: Cada element de l és la suma de k i l'element de L
a la seva mateixa posició */
{
    list<int>::iterator it;
    it = l.begin();
    /* Inv: ? */
    while (it != l.end()) {
        *it += k;
        ++it;
    }
}
```

Quin invariant podríem trobar?

Solució: sumar k a tots els elements d'una llista

```
void suma_llista_k(list<int> &l, int k)
/* Pre: l = L */
/* Post: Cada element de l és la suma de k i l'element de L
a la seva mateixa posició */
{
    list<int>::iterator it;
    it = l.begin();
    /* Inv: ? */
    while (it != l.end()) {
        *it += k;
        ++it;
    }
}
```

Quin invariant podríem trobar? / es modifica?

Solució: sumar k a tots els elements d'una llista

```

void suma_llista_k(list<int> &l, int k)
/* Pre: l = L */
/* Post: Cada element de l és la suma de k i l'element de L
a la seva mateixa posició */
{
    list<int>::iterator it;
    it = l.begin();
    /* Inv: ? */
    while (it != l.end()) {
        *it += k;
        ++it;
    }
}
    
```

Quin invariant podríem trobar? / es modifica? Què més és modifica?

Solució: sumar k a tots els elements d'una llista

Proposta d'invariant:

```

/* Inv: Cada element de l des de l'inici fins a
   la posició anterior a la que marca it és la suma
   de k més l'element corresponent d'L. Des d'it
   fins al final de l, els elements són iguals
   que els seus corresponents a L */
  
```

Justificació informal: sumar k a tots els elements d'una llista

1 *Inicialitzacions:*

Justificació informal: sumar k a tots els elements d'una llista

- 1 *Inicialitzacions:* Després d'executar `it = l.begin()` fem cert l'invariant.

Justificació informal: sumar k a tots els elements d'una llista

- 1 *Inicialitzacions:* Després d'executar `it = l.begin()` fem cert l'invariant.
- 2 *Condició de sortida:*

Justificació informal: sumar k a tots els elements d'una llista

- 1 *Inicialitzacions:* Després d'executar `it = l.begin()` fem cert l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tots els elems. de `l` siguin la suma del seu valor a `L` més `k`. Això s'aconsegueix quan `it` és igual a `l.end()`, per tant, ens quedem mentre `it != l.end()`.

Justificació informal: sumar k a tots els elements d'una llista

- 1 *Inicialitzacions:* Després d'executar `it = l.begin()` fem cert l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tots els elems. de `l` siguin la suma del seu valor a `L` més k . Això s'aconsegueix quan `it` és igual a `l.end()`, per tant, ens quedem mentre `it != l.end()`.
- 3 *Cos del bucle:*

Justificació informal: sumar k a tots els elements d'una llista

- 1 *Inicialitzacions:* Després d'executar `it = l.begin()` fem cert l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tots els elems. de `l` siguin la suma del seu valor a `L` més k . Això s'aconsegueix quan `it` és igual a `l.end()`, per tant, ens quedem mentre `it != l.end()`.
- 3 *Cos del bucle:* Si a l'inici d'una iteració qualsevol del bucle es compleixen l'invariant i la condició d'entrada en el bucle, després d'executar `*it+=k` i `++it` l'invariant torna a ser cert.

Justificació informal: sumar k a tots els elements d'una llista

- 1 *Inicialitzacions:* Després d'executar `it = l.begin()` fem cert l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tots els elems. de `l` siguin la suma del seu valor a `L` més k . Això s'aconsegueix quan `it` és igual a `l.end()`, per tant, ens quedem mentre `it != l.end()`.
- 3 *Cos del bucle:* Si a l'inici d'una iteració qualsevol del bucle es compleixen l'invariant i la condició d'entrada en el bucle, després d'executar `*it+=k` i `++it` l'invariant torna a ser cert.
- 4 *Acabament:*

Justificació informal: sumar k a tots els elements d'una llista

- 1 *Inicialitzacions:* Després d'executar `it = l.begin()` fem cert l'invariant.
- 2 *Condició de sortida:* Per satisfer la `Post` cal que tots els elems. de `l` siguin la suma del seu valor a `L` més k . Això s'aconsegueix quan `it` és igual a `l.end()`, per tant, ens quedem mentre `it != l.end()`.
- 3 *Cos del bucle:* Si a l'inici d'una iteració qualsevol del bucle es compleixen l'invariant i la condició d'entrada en el bucle, després d'executar `*it+=k` i `++it` l'invariant torna a ser cert.
- 4 *Acabament:* Agafem com a funció fita **la mida de la subllista** entre `it` i `l.end()`, perquè $\in \mathbb{N}$ i decreix a cada iteració.

Índex

- 1 Programes iteratius
- 2 Correctesa de programes
- 3 Justificació de la correctesa de programes iteratius
- 4 Exercici**

Exercici: cerca de dos elements iguals consecutius en una llista

- Donada una llista d'enters no buida, dissenyeu una funció iterativa que indiqui si la llista conté dos elements iguals consecutius.
- Feu servir aquesta especificació:

```
bool dos_seguits(const list<int> &l);
/* Pre: l no és buida */
/* Post: El resultat indica si l conté dos elements iguals
consecutius */
```

- Exemples:* Amb la llista $[-5, 5, 5, -2, 2, 5, -1]$ hauria de tornar cert, i amb la llista $[-5, 5, 6, -2, 2, 5, -1]$ hauria de tornar fals.